无锁并发队列

Posted on 2023-11-05 20:23:41 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2XkxkSnynSeJaEQtohQzWn20bSJ)

简介

前文介绍了如何通过内存顺序实现内存模型，本文基于前文的基础，利用内存顺序和内存模型的知识，带着大家探索无锁并发的应用，主要是通过无锁队列的实现来让大家熟悉无锁并发的实现方式。

环形队列

我们要实现无锁并发，经常会用到一种结构无锁队列，而无锁队列和我们经常使用的队列颇有不同，它采用的是环状的队列结构，为什么成环呢？主要有两个好处，一个是成环的队列大小是固定的，另外一个我们通过移动头和尾就能实现数据的插入和取出。

我们看下图是一个环形队列的基本结构

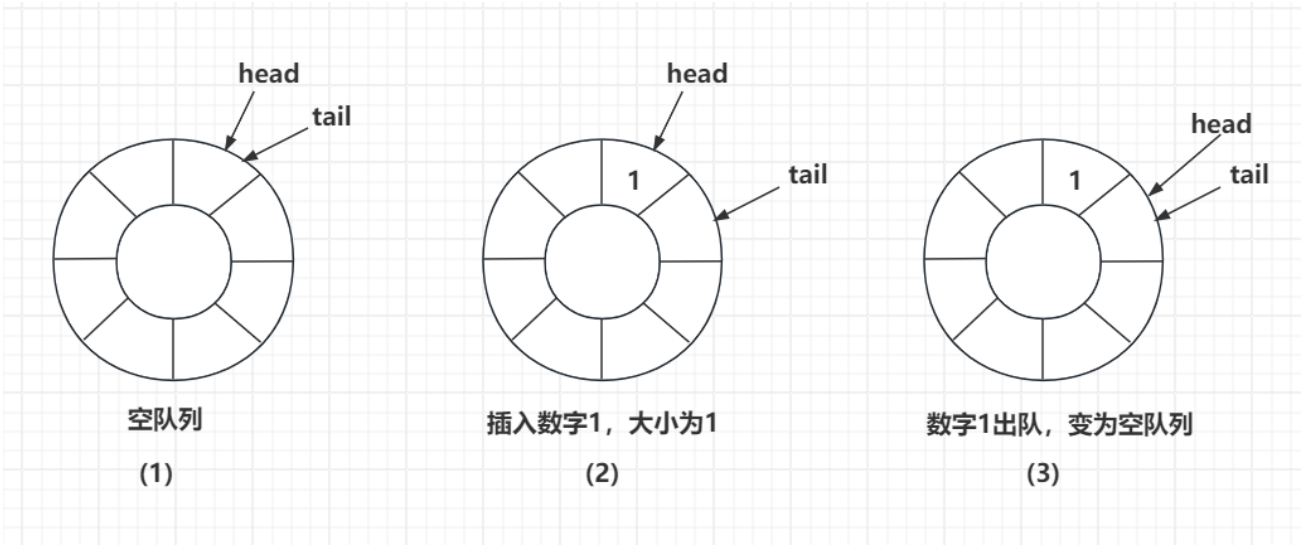


图1表示队列为空的时候，头节点和尾节点交会在一起，指向同一个扇区。

图2表示当我们你插入一个数字1后，队列大小为1，此时tail指针移动到下一个扇区，head指

向头部，1被存储在头部了。

图3表示当我们将数字1出队后，head指针向后移动一个扇区，此时head和tail指向同一个扇区，表示队列又为空了。那有人会问队列中数字1为什么不清空呢？其实不用清空，因为当我们插入新数据时就可以覆盖掉1这个无效的数据。

比如我们继续3图，连续插入几个数字，将队列填满。

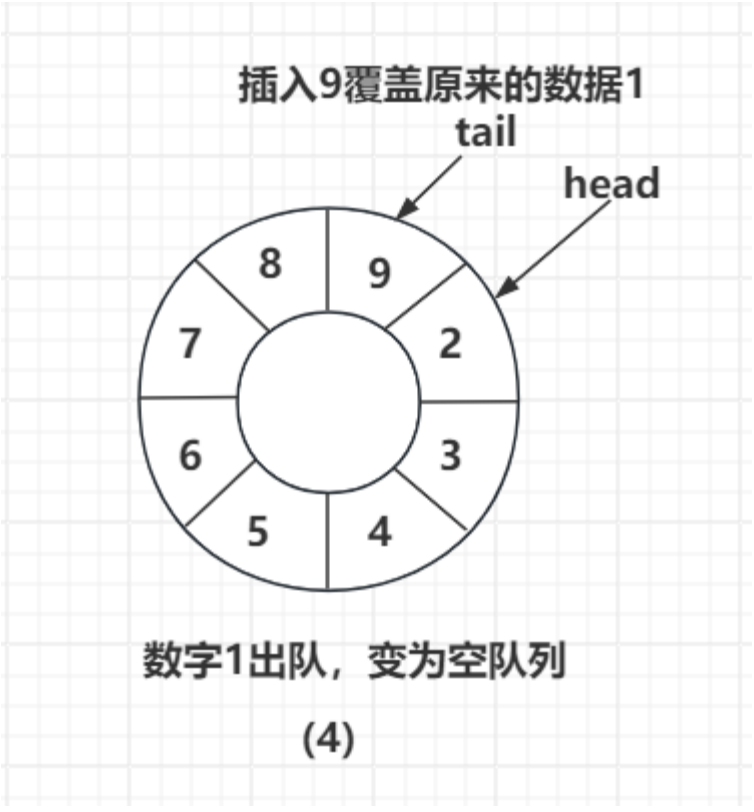


图4说明的就是当我们连续插入了几个数字，插入数据9的时候将原来1的数据覆盖了，所以环形队列删除数据的时候我们不用让数据出队，只要移动head指针即可。

另外我们从图4也能看出，此时tail指向的位置正好是head的前一个位置，这种情况表示队列满了。

用锁实现环形队列

我们可以用锁实现上述环形队列，在push和pop时分别加锁，并通过head和tail计算队列是否为满或者空。

代码比较简单，可以看看下面的写法

```cpp
#include <iostream>
#include <mutex>
#include <memory>

template<typename T, size_t Cap>
class CircularQueLk :private std::allocator<T> {
public:
    CircularQueLk() :_max_size(Cap + 1),_data(std::allocator<T>::allocate(_max_size)), _head(0), _tail(0) {}
    CircularQueLk(const CircularQueLk&) = delete;
    CircularQueLk& operator = (const CircularQueLk&) volatile = delete;
    CircularQueLk& operator = (const CircularQueLk&) = delete;

    ~CircularQueLk() {
        //循环销毁
        std::lock_guard<std::mutex>  lock(_mtx);
        //调用内部元素的析构函数
        while (_head != _tail) {
            std::allocator<T>::destroy(_data + _head);
            _head = (_head+1)%_max_size;
        }
        //调用回收操作
        std::allocator<T>::deallocate(_data, _max_size);
    }

    //先实现一个可变参数列表版本的插入函数最为基准函数
    template <typename ...Args>
    bool emplace(Args && ... args) {
        std::lock_guard<std::mutex> lock(_mtx);
        //判断队列是否满了
        if ((_tail + 1) % _max_size == _head) {
            std::cout << "circular que full ! " << std::endl;
            return false;
        }
        //在尾部位置构造一个T类型的对象，构造参数为args...
        std::allocator<T>::construct(_data + _tail, std::forward<Args>(args)...);
        //更新尾部元素位置
        _tail = (_tail + 1) % _max_size;
        return true;
    }

    //push 实现两个版本，一个接受左值引用，一个接受右值引用
```

```
42.
43.     //接受左值引用版本
44.     bool push(const T& val) {
45.         std::cout << "called push const T& version" << std::endl;
46.         return emplace(val);
47.     }
48.
49.     //接受右值引用版本，当然也可以接受左值引用，T&&为万能引用
50.     // 但是因为我们实现了const T&
51.     bool push(T&& val) {
52.         std::cout << "called push T&& version" << std::endl;
53.         return emplace(std::move(val));
54.     }
55.
56.     //出队函数
57.     bool pop(T& val) {
58.         std::lock_guard<std::mutex> lock(_mtx);
59.         //判断头部和尾部指针是否重合，如果重合则队列为空
60.         if (_head == _tail) {
61.             std::cout << "circular que empty ! " << std::endl;
62.             return false;
63.         }
64.         //取出头部指针指向的数据
65.         val = std::move(_data[_head]);
66.         //更新头部指针
67.         _head = (_head + 1) % _max_size;
68.         return true;
69.     }
70. private:
71.     size_t _max_size;
72.     T* _data;
73.     std::mutex _mtx;
74.     size_t _head = 0;
75.     size_t _tail = 0;
76. };
```

测试也比较简单，我们写一个函数，初始化队列大小为5，测试队列push满的情况和pop直到为空的情况

```
void TestCircularQue() {
    //最大容量为10
    CircularQueLk<MyClass, 5> cq_lk;
    MyClass mc1(1);
    MyClass mc2(2);
    cq_lk.push(mc1);
    cq_lk.push(std::move(mc2));
    for (int i = 3; i <= 5; i++) {
        MyClass mc(i);
        auto res = cq_lk.push(mc);
        if (res == false) {
            break;
        }
    }

    cq_lk.push(mc2);

    for (int i = 0; i < 5; i++) {
        MyClass mc1;
        auto res = cq_lk.pop(mc1);
        if (!res) {
            break;
        }
        std::cout << "pop success, " << mc1 << std::endl;
    }

    auto res = cq_lk.pop(mc1);
}
```

结果如下

```
called push const T& version
called push T&& version
called push const T& version
called push const T& version
called push const T& version
called push const T& version
circular que full !

pop success, MyClass Data is 1
pop success, MyClass Data is 2
pop success, MyClass Data is 3
pop success, MyClass Data is 4
pop success, MyClass Data is 5
circular que empty !
```

无锁队列

那如果我们用原子变量而不是用锁实现环形队列，那就是无锁并发的队列了。还记得我们之前提到的原子变量的读改写操作吗?

```
bool std::atomic<T>::compare_exchange_weak(T &expected, T desired);
bool std::atomic<T>::compare_exchange_strong(T &expected, T desired);
```

`compare_exchange_strong` 会比较原子变量 `atomic<T>` 的值和 `expected` 的值是否相等，如果相等则执行交换操作，将 `atomic<T>` 的值换为 `desired` 并且返回true,否则将expected的值修改为bool变量的值，并且返回false.

其伪代码可以这么理解

```
template <typename T>
bool atomic<T>::compare_exchange_strong(T &expected, T desired) {
    std::lock_guard<std::mutex> guard(m_lock);
    if (m_val == expected)
        return m_val = desired, true;
    else
        return expected = m_val, false;
}
```

`compare_exchange_weak` 功能比 `compare_exchange_strong` 弱一些，他不能保

证 `atomic<T>` 的值和 `expected` 的值相等时也会做交换，很可能原子变量和预期值相等也会返回false，所以使用要多次循环使用。

我们们定义一个类CircularQueSeq, 其内容和之前我们定义的类CircularQueLk差不多，只不过将类的成员变量mutex换成atomic类型的原子变量, 我们可以利用自旋锁的思路将锁替换为原子变量循环检测的方式，进而达到锁住互斥逻辑的效果。

大家可以先看一下全部的代码感受一下

```
1. template<typename T, size_t Cap>
2. class CircularQueSeq :private std::allocator<T> {
3. public:
4.     CircularQueSeq() :_max_size(Cap + 1), _data(std::allocator<T>::allocate(_max_size)), _ato
   mic_using(false),_head(0), _tail(0) {}
5.     CircularQueSeq(const CircularQueSeq&) = delete;
6.     CircularQueSeq& operator = (const CircularQueSeq&) volatile = delete;
7.     CircularQueSeq& operator = (const CircularQueSeq&) = delete;
8.
9.     ~CircularQueSeq() {
10.        //循环销毁
11.        bool use_expected = false;
12.        bool use_desired = true;
13.        do
14.        {
15.            use_expected = false;
16.            use_desired = true;
17.        }
18.        while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
19.        //调用内部元素的析构函数
20.        while (_head != _tail) {
21.            std::allocator<T>::destroy(_data + _head);
22.            _head =  (_head+1) % _max_size;
23.        }
24.        //调用回收操作
25.        std::allocator<T>::deallocate(_data, _max_size);
26.
27.        do
28.        {
29.            use_expected = true;
30.            use_desired = false;
31.        }
32.        while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
33.    }
34.
35.    //先实现一个可变参数列表版本的插入函数最为基准函数
36.    template <typename ...Args>
37.    bool emplace(Args && ... args) {
38.
39.        bool use_expected = false;
40.        bool use_desired = true;
41.        do
```

```
42.     {
43.         use_expected = false;
44.         use_desired = true;
45.     }
46.     while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
47.
48.     //判断队列是否满了
49.     if ((_tail + 1) % _max_size == _head) {
50.         std::cout << "circular que full ! " << std::endl;
51.         do
52.         {
53.             use_expected = true;
54.             use_desired = false;
55.         }
56.         while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
57.         return false;
58.     }
59.     //在尾部位置构造一个T类型的对象，构造参数为args...
60.     std::allocator<T>::construct(_data + _tail, std::forward<Args>(args)...);
61.     //更新尾部元素位置
62.     _tail = (_tail + 1) % _max_size;
63.
64.     do
65.     {
66.         use_expected = true;
67.         use_desired = false;
68.     }
69.     while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
70.
71.     return true;
72. }
73.
74. //push 实现两个版本，一个接受左值引用，一个接受右值引用
75.
76. //接受左值引用版本
77. bool push(const T& val) {
78.     std::cout << "called push const T& version" << std::endl;
79.     return emplace(val);
80. }
81.
82. //接受右值引用版本，当然也可以接受左值引用，T&&为万能引用
83. // 但是因为我们实现了const T&
```

```
84.     bool push(T&& val) {
85.         std::cout << "called push T&& version" << std::endl;
86.         return emplace(std::move(val));
87.     }
88.
89.     //出队函数
90.     bool pop(T& val) {
91.
92.         bool use_expected = false;
93.         bool use_desired = true;
94.         do
95.         {
96.             use_desired = true;
97.             use_expected = false;
98.         } while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
99.         //判断头部和尾部指针是否重合，如果重合则队列为空
100.        if (_head == _tail) {
101.            std::cout << "circular que empty ! " << std::endl;
102.            do
103.            {
104.                use_expected = true;
105.                use_desired = false;
106.            }
107.            while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
108.            return false;
109.        }
110.        //取出头部指针指向的数据
111.        val = std::move(_data[_head]);
112.        //更新头部指针
113.        _head = (_head + 1) % _max_size;
114.
115.        do
116.        {
117.            use_expected = true;
118.            use_desired = false;
119.        }while (!_atomic_using.compare_exchange_strong(use_expected, use_desired));
120.        return true;
121.    }
122. private:
123.    size_t _max_size;
124.    T* _data;
125.    std::atomic<bool> _atomic_using;
```

```
126.     size_t _head = 0;
127.     size_t _tail = 0;
128. };
```

我们可以看到 emplace 函数以及 pop 函数等将锁替换为原子变量。采用 do while 的方式就是因为 compare_exchange_strong 比较原子变量和 use_expected 的值不同的时候会使 use_expected 改变，所以我们需要在再次循环之前重置 use_expected 和 use_desired 的值。

我们可以写一个函数在单线程情况下下测试一下

```
1. void TestCircularQueSeq()
2. {
3.     CircularQueSeq<MyClass, 3> cq_seq;
4.     for(int i = 0; i < 4; i++)
5.     {
6.         MyClass mc1(i);
7.         auto res = cq_seq.push(mc1);
8.         if(!res)
9.         {
10.            break;
11.        }
12.    }
13.
14.    for(int i = 0; i < 4; i++)
15.    {
16.        MyClass mc1;
17.        auto res = cq_seq.pop(mc1);
18.        if(!res)
19.        {
20.            break;
21.        }
22.
23.        std::cout << "pop success, " << mc1 << std::endl;
24.    }
25.
26.    for (int i = 0; i < 4; i++)
27.    {
28.        MyClass mc1(i);
29.        auto res = cq_seq.push(mc1);
30.        if (!res)
31.        {
32.            break;
33.        }
34.    }
35.
36.    for (int i = 0; i < 4; i++)
37.    {
38.        MyClass mc1;
39.        auto res = cq_seq.pop(mc1);
40.        if (!res)
41.        {
42.            break;
```

```
        }

        std::cout << "pop success, " << mc1 << std::endl;
    }
}
```

运行结果

```
called push const T& version
called push const T& version
called push const T& version
called push const T& version
circular que full !
pop success, MyClass Data is 0
pop success, MyClass Data is 1
pop success, MyClass Data is 2
circular que empty !
called push const T& version
called push const T& version
called push const T& version
called push const T& version
circular que full !
pop success, MyClass Data is 0
pop success, MyClass Data is 1
pop success, MyClass Data is 2
circular que empty !
```

多线程情况下也能保证安全是因为原子变量循环检测保证有且只有一个线程修改成功。读取也是这样。

单一原子变量的弊端

我们考虑上述单一原子变量的弊端

多个线程push和pop操作耦合读太高，同一时刻仅有一个线程pop或者push，而且互斥逻辑的精度不够。影响效率。

我们需要考虑将pop和push操作解耦，我们采用的是环形队列，将 `tail` 和 `head` 作为原子变量可以实现精细控制。

比如我们做push操作的时候，一个线程更新万 `tail` 标签和数据后，其他线程就可以pop或者push了，精细控制的好处就是效率提升。

我们定义一个新的类CircularQueLight，类的基本数据结构和CircularQueSeq差不多。

```
1. template<typename T, size_t Cap>
2. class CircularQueLight: private std::allocator<T>
3. {
4. public:
5.     CircularQueLight():_max_size(Cap + 1),
6.     _data(std::allocator<T>::allocate(_max_size))
7.     , _head(0), _tail(0) {}
8.
9.     CircularQueLight(const CircularQueLight&) = delete;
10.    CircularQueLight& operator = (const CircularQueLight&) volatile = delete;
11.    CircularQueLight& operator = (const CircularQueLight&) = delete;
12. private:
13.    size_t _max_size;
14.    T* _data;
15.    std::atomic<size_t>  _head;
16.    std::atomic<size_t> _tail;
17. };
```

我们将 `_head` 和 `_tail` 替换为原子变量。

接下来我们考虑 `pop` 逻辑

```
bool pop(T& val) {

    size_t h;
    do
    {
        h = _head.load();  //1 处
        //判断头部和尾部指针是否重合，如果重合则队列为空
        if(h == _tail.load())
        {
            return false;
        }
        val = _data[h]; // 2处

    } while (!_head.compare_exchange_strong(h,
        (h+1)% _max_size)); //3 处

    return true;
}
```

在pop逻辑里我们在1处load获取头部head的值，在2处采用了复制的方式将头部元素取出赋值给val，而不是通过 `std::move` ，因为多个线程同时 `pop` 最后只有一个线程成功执行3处代码退出，而失败的则需要继续循环，从更新后的head处 `pop` 元素。所以不能用 `std::move` ，否则会破坏原有的队列数据。

接下来我们来做push的函数逻辑

```
bool push(T& val)
{
    size_t t;
    do
    {
        t = _tail.load(); //1
        //判断队列是否满
        if( (t+1)%_max_size == _head.load())
        {
            return false;
        }

        _data[t] = val; //2

    } while (!_tail.compare_exchange_strong(t,
        (t + 1) % _max_size)); //3

    return true;
}
```

push函数的逻辑乍一看和pop一样，但是我们会发现多线程push的情况存在线程安全问题。

比如我们线程1 `push(1)` 而线程2 `push(2)` . 很有可能的顺序是

1.1 -> 1.2 -> 2.1 -> 2.2 -> 1.3

这样我们看到的效果就是 `_data[t]` 被存储为2了，而实际情况应该是被存储为1，因为线程1的原子变量生效，而线程2的原子变量不满足需继续循环。所以 `_data[t]` 必须修改为1.

那我们改进一下 `push` 的函数

```
bool push(T& val)
 {
    size_t t;
    do
    {
       t = _tail.load();  //1
       //判断队列是否满
       if( (t+1)%_max_size == _head.load())
       {
          return false;
       }



    } while (!_tail.compare_exchange_strong(t,
       (t + 1) % _max_size));  //3

    _data[t] = val; //2

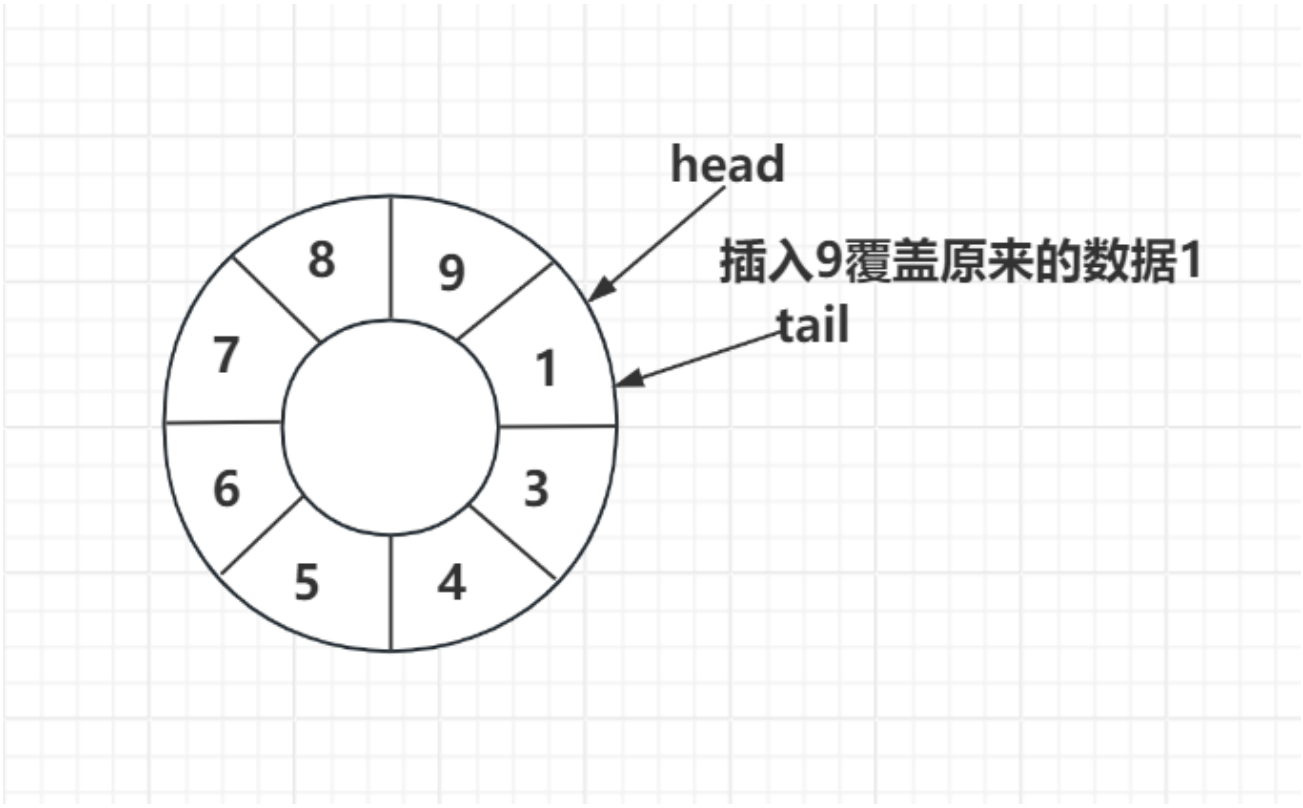
    return true;
 }
```

我们将2处的代码移动到循环之外，这样能保证多个线程push，仅有一个线程生效时，他写入的数据一定是本线程要写入到tail的数据，而此时tail被缓存在t里，那是一个线程本地变量，所以在这种情况下我们能确定即使多个线程运行到2处，他们的t值也是不同的，并不会产生线程安全问题。

毕竟多个线程push数据时对资源的竞争仅限tail。

但是这种push操作仍然会有安全问题

我们思考这种情况



此时head和tail都指向1这个位置，当我们执行 `push(9)` 时，按照我们的逻辑会先执行3再执行2.

也就是会先将tail移动，然后更新1的值为9.

那如果我们更新了tail之后，还没来的及更新1为9，那么此时如果有其他的线程读取head的值，会读取到1，而不是9.

从多线程安全角度来讲这是不安全的，我们理想的情况是一个线程写完数据后另一个线程读取的就是之前写入的最新值。

为了解决这个问题，我们可以增加另一个原子变量 `_tail_update` 来标记尾部数据是否修改完成，如果尾部数据没有修改完成，此时其他线程pop时获取的数据就是不安全的，所以pop要返回false。

先实现push版本

```
bool push(const T& val)
{
    size_t t;
    do
    {
        t = _tail.load();  //1
        //判断队列是否满
        if( (t+1)%_max_size == _head.load())
        {
            return false;
        }



    } while (!_tail.compare_exchange_strong(t,
        (t + 1) % _max_size));  //3

    _data[t] = val; //2
    size_t tailup;
    do
    {
        tailup = t;

    } while (_tail_update.compare_exchange_strong(tailup,
        (tailup + 1) % _max_size));
    return true;
}
```

再实现pop版本

```
bool pop(T& val) {

    size_t h;
    do
    {
        h = _head.load();  //1 处
        //判断头部和尾部指针是否重合，如果重合则队列为空
        if(h == _tail.load())
        {
            return false;
        }

        //判断如果此时要读取的数据和tail_update是否一致，如果一致说明尾部数据未更新完
        if(h == _tail_update.load())
        {
            return false;
        }
        val = _data[h]; // 2处

    } while (!_head.compare_exchange_strong(h,
        (h+1)% _max_size)); //3 处

    return true;
}
```

pop版本也是，先判断队列是否为空，再判断h是否和_tail_update的值相等，如果相等说明有写数据的没更新完，直接返回false或者循环等待也行，为了方便我们直接返回false即可。

因为我们知道原子操作默认采用的是 `memory_order_seq_cst` 内存顺序，性能上不是最优的，我们可以用 `acquire` 和 `release` 的内存顺序实现同步的效果。

优化性能

我们用acquire和release模型优化上述代码，实现同步。
最简单的方式就是将load的地方变为 `memory_order_relaxed` ，compare_exchange_strong的地方变为 `memory_order_release`

我们先看pop操作

```
bool pop(T& val) {

    size_t h;
    do
    {
        h = _head.load(std::memory_order_relaxed);  //1 处
        //判断头部和尾部指针是否重合，如果重合则队列为空
        if (h == _tail.load(std::memory_order_acquire)) //2处
        {
            std::cout << "circular que empty ! " << std::endl;
            return false;
        }

        //判断如果此时要读取的数据和tail_update是否一致，如果一致说明尾部数据未更新完
        if (h == _tail_update.load(std::memory_order_acquire)) //3处
        {
            return false;
        }
        val = _data[h]; // 2处

    } while (!_head.compare_exchange_strong(h,
        (h + 1) % _max_size, std::memory_order_release, std::memory_order_relaxed)); //4处
    std::cout << "pop data success, data is " << val << std::endl;
    return true;
}
```

1 处为 `memory_order_relaxed` 是因为即使多个线程pop，每个线程获取的head可能不及时，这个没关系，因为我们有4处的while来重试。

2 compare_exchange_strong操作，在期望的条件匹配时采用 `memory_order_release` , 期望的条件不匹配时 `memory_order_relaxed` 可以提升效率，毕竟还是要重试的。

我们再看push 操作

```
bool push(const T& val)
{
    size_t t;
    do
    {
        t = _tail.load(std::memory_order_relaxed);  //5
        //判断队列是否满
        if ((t + 1) % _max_size == _head.load(std::memory_order_acquire))
        {
            std::cout << "circular que full ! " << std::endl;
            return false;
        }



    } while (!_tail.compare_exchange_strong(t,
        (t + 1) % _max_size, std::memory_order_release, std::memory_order_relaxed));  //6

    _data[t] = val;
    size_t tailup;
    do
    {
        tailup = t;

    } while (_tail_update.compare_exchange_strong(tailup,
        (tailup + 1) % _max_size, std::memory_order_release, std::memory_order_relaxed));
//7

    std::cout << "called push data success " << val << std::endl;
    return true;
}
```

两个线程协同工作，一个线程先push，另一个线程后pop，那么对于tail部分和_tail_update，我们要保证push的结果 `_data[t] = val;` 先于pop的结果 `val = _data[h];`

所以push线程中对于_tail_update的compare_exchange_strong操作采用 `memory_order_release` 方式。

pop线程对于_tail_update的load操作采用 `memory_order_acquire` 。

如果一个线程先pop，另一个线程先push，那么对于head部分，我们要保证pop的结果 `val = _data[h];` 先于pop的结果 `_data[t] = val;` 。

思考

优势

无锁高并发. 虽然存在循环重试, 但是这只会在相同操作并发的时候出现. push 不会因为与 pop 并发而重试, 反之亦然.

缺陷

这样队列只应该存储标量, 如果存储类对象时，多个push线程只有一个线程push成功，而拷贝复制的开销很大，其他线程会循环重试，每次重试都会有开销。

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day12-circularque (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day12-circularque)

浏览(2203) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(537) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)